Project_House_Price_Predictor

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
# load data
df=pd.read_csv('HousePricePrediction.csv')
df.head(5)
```

Out[2]:

	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt
0	0	60	RL	8450	Inside	1Fam	5	2003
1	1	20	RL	9600	FR2	1Fam	8	1976
2	2	60	RL	11250	Inside	1Fam	5	2001
3	3	70	RL	9550	Corner	1Fam	5	1915
4	4	60	RL	14260	FR2	1Fam	5	2000

In [3]:
```
df.shape
```

Out[3]:
```
(2919, 13)
```

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2919 entries, 0 to 2918
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Id            2919 non-null   int64
 1   MSSubClass    2919 non-null   int64
 2   MSZoning      2915 non-null   object
 3   LotArea       2919 non-null   int64
 4   LotConfig     2919 non-null   object
 5   BldgType      2919 non-null   object
 6   OverallCond   2919 non-null   int64
 7   YearBuilt     2919 non-null   int64
 8   YearRemodAdd  2919 non-null   int64
 9   Exterior1st   2918 non-null   object
 10  BsmtFinSF2    2918 non-null   float64
 11  TotalBsmtSF   2918 non-null   float64
 12  SalePrice     1460 non-null   float64
dtypes: float64(3), int64(6), object(4)
memory usage: 296.6+ KB
```

In [5]:
```
df.describe()
```

Out[5]:

	Id	MSSubClass	LotArea	OverallCond	YearBuilt	YearRemodA
count	2919.000000	2919.000000	2919.000000	2919.000000	2919.000000	2919.000(
mean	1459.000000	57.137718	10168.114080	5.564577	1971.312778	1984.264
std	842.787043	42.517628	7886.996359	1.113131	30.291442	20.894
min	0.000000	20.000000	1300.000000	1.000000	1872.000000	1950.000(
25%	729.500000	20.000000	7478.000000	5.000000	1953.500000	1965.000(
50%	1459.000000	50.000000	9453.000000	5.000000	1973.000000	1993.000(
75%	2188.500000	70.000000	11570.000000	6.000000	2001.000000	2004.000(
max	2918.000000	190.000000	215245.000000	9.000000	2010.000000	2010.000(

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Id                 0
MSSubClass         0
MSZoning           4
LotArea            0
LotConfig          0
BldgType           0
OverallCond        0
YearBuilt          0
YearRemodAdd       0
Exterior1st        1
BsmtFinSF2         1
TotalBsmtSF        1
SalePrice       1459
dtype: int64
```

In [7]:
```
# Data Preprocessing - Data cleaning
df.drop(['Id'],axis=1,inplace=True)       # Drop Id as it doesn't contribute in
```

In [8]:
```
# Replacing SalePrice empty values with their mean values
df['SalePrice'].isnull().sum()

df['SalePrice'] = df['SalePrice'].fillna(df['SalePrice'].mean())

df.isnull().sum()
```

Out[8]:
```
MSSubClass      0
MSZoning        4
LotArea         0
LotConfig       0
BldgType        0
OverallCond     0
YearBuilt       0
YearRemodAdd    0
Exterior1st     1
BsmtFinSF2      1
TotalBsmtSF     1
SalePrice       0
dtype: int64
```

In [9]:
```
# Drop records with null values

df=df.dropna()
df.isnull().sum()
df.head()
```

Out[9]:

	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt	Year
0	60	RL	8450	Inside	1Fam	5	2003	
1	20	RL	9600	FR2	1Fam	8	1976	
2	60	RL	11250	Inside	1Fam	5	2001	
3	70	RL	9550	Corner	1Fam	5	1915	
4	60	RL	14260	FR2	1Fam	5	2000	

In [10]:
```
# Data Preprocessing - Encoding
cols = ['MSZoning', 'LotConfig', 'BldgType', 'Exterior1st']
df = pd.get_dummies(df, columns=cols, drop_first=True)
```

In [11]:
```
# Train-Test Split
from sklearn.model_selection import train_test_split

x=df.drop(['SalePrice'],axis=1)
y=df['SalePrice']
```

In [12]:
```
x=df.drop(['SalePrice'],axis=1)
y=df['SalePrice']

x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)
x_train.shape,x_test.shape,y_train.shape,y_test.shape
```

Out[12]:
```
((2330, 33), (583, 33), (2330,), (583,))
```

In [13]:
```
x_train.head()
```

Out[13]:

	MSSubClass	LotArea	OverallCond	YearBuilt	YearRemodAdd	BsmtFinSF2	Totall
1453	20	17217	5	2006	2006	0.0	
2021	20	19508	5	1974	1974	0.0	
188	90	7018	5	1979	1979	0.0	
1273	80	11512	7	1959	2006	0.0	
1960	20	7560	5	1971	1971	613.0	

5 rows × 33 columns

In [16]:
```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model
```

Out[16]:

```
▾ LinearRegression
LinearRegression()
```

In [17]:

```
model.fit(x_train,y_train)
```

Out[17]:

```
▾ LinearRegression
LinearRegression()
```

In [28]:

```
# Model Evaluation

from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
from sklearn.metrics import mean_absolute_percentage_error
import numpy as np

y_pred=model.predict(x_test)

print("R2 Score:",r2_score(y_test,y_pred))
print("mean : ",mean_absolute_error(y_test,y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print(mean_absolute_percentage_error(y_test, y_pred))
```

```
R2 Score: 0.3741421796138331
mean :  30829.936664322337
RMSE: 41138.558702301714
0.1874202546342832
```

In [36]:

```
# Feature Scaling - to try & improve baseline performance

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

model = LinearRegression()
model.fit(x_train_scaled, y_train)

y_pred = model.predict(x_test_scaled)

print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print(mean_absolute_percentage_error(y_test, y_pred))
```

```
R2 Score: 0.3741421796138531
MAE: 30829.93666432187
RMSE: 41138.55870230106
0.18742025463427797
```

In [ ]:

In [ ]: